

IES CHAN DO MONTE

C.S. de Desarrollo de Aplicaciones Multiplataforma

Modulo Acceso a datos

ACTIVIDAD 3: Operaciones de Actualización

Objetivos:

Na actividade que nos ocupa aprenderanse os seguintes conceptos e manexo de destrezas:

- **Realizar operacións de modificación de datos** sobre unha base de datos relacional empregando sentenzas SQL de inserción, actualización e eliminación.
- **Obter a clave primaria xerada automaticamente** polo SGBD tras unha inserción.
- **Modificar rexistros directamente a través dun ResultSet dinámico**, utilizando modos TYPE_SCROLL_SENSITIVE e CONCUR_UPDATABLE.
- **Navegar dentro dun ResultSet** empregando métodos de desprazamento (next, previous, first, last, absolute, relative) para comprender e controlar o acceso á información.
- **Aplicar transaccións** en operacións encadeadas de actualización, garantindo a consistencia da base de datos mediante commit() e rollback().
- **Integrar procesos por lotes (batch)** dentro dunha transacción para mellorar o rendemento.
- **Xestionar excepcións SQL** derivadas de operacións de actualización e transaccións, garantindo robustez e claridade na execución.
- **Aplicar boas prácticas de deseño orientado a obxectos**, asegurando responsabilidade única, separación de capas e mantenibilidade do código.
- **Probar e documentar as aplicacións desenvolvidas**, validando que os cambios realizados na base de datos son correctos e que os recursos se liberan adecuadamente.

Metodoloxía aplicada

Antes de iniciar os exercicios, débese seguir a mesma metodoloxía de deseño aplicada ata o momento:

- **Principio de responsabilidade única:** cada clase debe ter unha función clara (clases -modelo-), lóxica, persistencia).
- **Uso de clases modelo:** representar as entidades da base de datos (Departamento, Empregado, Proxecto, etc-) como obxectos Java.
- **Separación de capas:**
 - Capa de persistencia → executa a consulta SQL.
 - Capa de lóxica → chama á persistencia, procesa os resultados, validación e mostra os resultados

Proyecto nuevo

Crea un novo proxecto chamado: A3UD2-xxx, onde xxx corresponde ás iniciais do teu nome e apelidos.

Exercicio 1.

Crea un método en Java que permita inserir un familiar dun empregado, teniendo en cuenta que:

O número do familiar debe ser secuencial por empregado: Comeza en 1 e incrementase 1, 2, 3... segundo os familiares xa rexistrados para ese empregado.

Non se pode rexistrar o mesmo familiar dúas veces para o mesmo empregado

Debe visualizar mensaxes de erro representativas. Se ocorre un erro SQL, debe mostrarse unha mensaxe clara e comprensible.

Ejercicio 2.

Realiza método que permita insertar un nuevo vehículo (Renting o propio), guardando la información en las tablas correspondientes.

El número de matrícula es único.

Haz llamadas al método para insertar la siguiente información:

Vehiculopropio:

Codigo	Matricula	Marca	Modelo	Tipo	FechaCompra	Precio
1	1234ABC	Toyota	Corolla	G	2023-05-15	15000

VehiculoRenting

Codigo	Matricula	Marca	Modelo	Tipo	FechaInicio	PrecioMensual	Meses
2	5678XYZ	Ford	Fiesta	D	2023-07-01	200	24

NOTA:

- Los datos **char** y **varchar de SQL**, en las sentencias parametrizadas se utiliza `setString`. Debemos tener cuidado si en la BD tenemos datos `char(1)`, y en java, en la clase tenemos un campo como `char`, tenemos que convertir el campo `char` a `String` para utilizarlo en la sentencia parametrizada y darle valor con `setString`. Ejemplo; si color es `char`,

```
sentencia.setString(7, String.valueOf(foto.getColor()));
```

- Acuérdate que el **operador instanceof nos** permite comprobar si un objeto es de una clase concreta, te va a servir de ayuda al insertar un tipo de vehículo (si es propio o de Renting).
- El Microsoft JDBC Driver para SQL Server admite las API de JDBC 3.0 opcionales para recuperar los identificadores de fila generados automáticamente. El valor principal de esta característica es proporcionar un método de modo que los valores `IDENTITY` estén disponibles para la aplicación que actualiza la tabla de base de datos

El conjunto de resultados devuelto por el método `getGeneratedKeys` de la clase `Statement` contiene tan solo una columna y se devuelve el nombre de columna `GENERATED_KEYS`. Si las claves generadas se solicitan en una tabla que no contiene la columna `IDENTITY`, el controlador JDBC devuelve un conjunto de resultados con valores nulos.

Ejemplo

```
CREATE TABLE Ejemplo
```

```
(Col1 int IDENTITY,
```

```
Col2 varchar(40),
```

```
Col3 int);
```

```
public static void Insertar(Connection con) {
    try(
        Statement stmt = con.createStatement(); {
            String SQL = "INSERT INTO Ejemplo (Col2, Col3) VALUES ('Ejemplo', 50)";
            int count = stmt.executeUpdate(SQL, Statement.RETURN_GENERATED_KEYS);
            ResultSet rs = stmt.getGeneratedKeys();
            If(rs.next()){
                int clave = rs.getInt(1);
                System.out.println("clave " + clave);
            }

            catch (SQLException e) {
                e.printStackTrace();
            }
        }
```

Exercicio 3.

Realiza un método en Java que permita **cambiar o departamento que controla un proxecto**, cumprindo as seguintes condicións:

- O método recibirá como parámetros:
 - O **nome do departamento**.
 - O **nome do proxecto**.
- Se o departamento non existe, deberá mostrarse unha **mensaxe de erro clara e representativa**.
- Deben empregarse **sentenzas pre-compiladas (PreparedStatement)**.
- Débese controlar adecuadamente calquera **excepción SQL**, mostrando mensaxes comprensibles para o usuario.

Exercicio 4.

Realiza un método en Java que permita **eliminar un proxecto da base de datos**.

O método recibirá como parámetro:

- O **número identificador do proxecto**.

Condicións que se deben cumprir

- O proxecto debe existir na base de datos.
- **Antes de borrar o proxecto**, deberán imprimirse **todos os datos do proxecto** e o **NSS e nome completo de todos os empregados asignados a ese proxecto**.
- As operacións deberán realizarse de forma controlada para garantir a **consistencia da base de datos**.
- Deben empregarse **sentenzas pre-compiladas (PreparedStatement)**.
- Débese controlar calquera **excepción SQL**, mostrando mensaxes de erro claras e comprensibles.

Exercicio 5.

Crea un método en Java que permita incrementar o salario dos empregados fixos da base de datos.

O método recibirá como parámetros:

- Un valor incremento (decimal) que indica canto se debe sumar ao salario actual.
- Unha lista de NSS dos empregados aos que se lles aplicará o incremento.

O método deberá realizar as actualizacións utilizando **PreparedStatement** e **batch update** para optimizar a execución.

Se se produce algún erro, se reverta a actualización de todos os empregados.

Debe controlarse calquera excepción **SQLException**, imprimindo mensaxes claras de erro.

Ao finalizar correctamente a operación, deberá imprimirse un mensaxe indicando o número de empregados actualizados.

Exercicio 6.

Crea un método en Java que reciba como parámetro un obxecto Proxecto e insira os seus datos na táboa PROXECTO. O obxecto Proxecto contén os datos dun proxecto novo (número, nome, lugar e número do departamento que o controla).

A inserción realizarase **a través dun ResultSet dinámico** (TYPE_SCROLL_SENSITIVE, CONCUR_UPDATABLE) xerado mediante unha consulta que devolva todos os datos da táboa PROXECTO.

Antes de realizar a inserción, o método debe:

1. Comprobar que **o número e o nome do proxecto non existen** xa na táboa.
 - Para iso, crear un método auxiliar que devolva true se o proxecto existe e false no caso contrario.
2. Comprobar que **o número do departamento que controla o proxecto existe** na táboa DEPARTAMENTO.
 - Para iso, crear un método auxiliar que devolva true se o departamento existe e false no caso contrario.

contrario.

O método debe xestionar correctamente os erros, mostrando mensaxes claras

Exercicio 7.

Crea un método que reciba como parámetros:

- Unha cantidade (incremento)
- Un número de departamento

O método debe incrementar o **salario** de todos os empregados dese departamento en esa cantidade.

- A actualización realizarase mediante **ResultSet dinámico** (modo actualización).
- Controla as excepcións SQL e garante a consistencia da base de datos.

Exercicio 8.

Crea un método que execute unha consulta parametrizada (PreparedStatement) para obter:

- NSS, Nome completo (nome + apelidos), Localidade e Salario

Dos empregados que teñan asignado un número de proxectos **maior que o parámetro introducido**.

O ResultSet obtido debe ser de **só lectura** e con **scroll** para permitir navegar en todas as direccións. A partir del, realiza o seguinte:

1. Visualiza a información da **primeira fila** do ResultSet.
2. Visualiza a información da **última fila**.
3. Visualiza a información da **antepenúltima fila**.
4. Recórreo en sentido **inverso**, desde a última ata a primeira fila, mostrando todos os datos.